

Arquitectura de Software — Unidad 1

Actividad práctica — Tema 2: Principios de diseño y POO como base arquitectónica

Fundamentos esenciales de POO

Abstracción

La abstracción consiste en modelar las entidades relevantes de un dominio resaltando sólo sus características esenciales y omitiendo detalles de implementación irrelevantes. Permite definir interfaces y clases que representen conceptos del mundo real sin exponer la complejidad interna.

Encapsulamiento

El encapsulamiento agrupa datos y comportamientos relacionados dentro de una unidad (clase) y oculta los detalles internos mediante modificadores de acceso (private/protected). Protege el estado interno y obliga a usar métodos controlados para interactuar con el objeto, facilitando mantenimiento y evolución.

Herencia

La herencia permite crear nuevas clases (subclases) que reutilizan y especializan el comportamiento de una clase base (superclase). Facilita la reutilización de código y la expresión de relaciones "es-un" entre entidades.

Polimorfismo

El polimorfismo permite tratar instancias de distintas clases de manera uniforme a través de una interfaz común (por ejemplo, mediante clases base o interfaces). Facilita la extensión del sistema sin cambiar el código cliente.

Ejemplo simple en Java

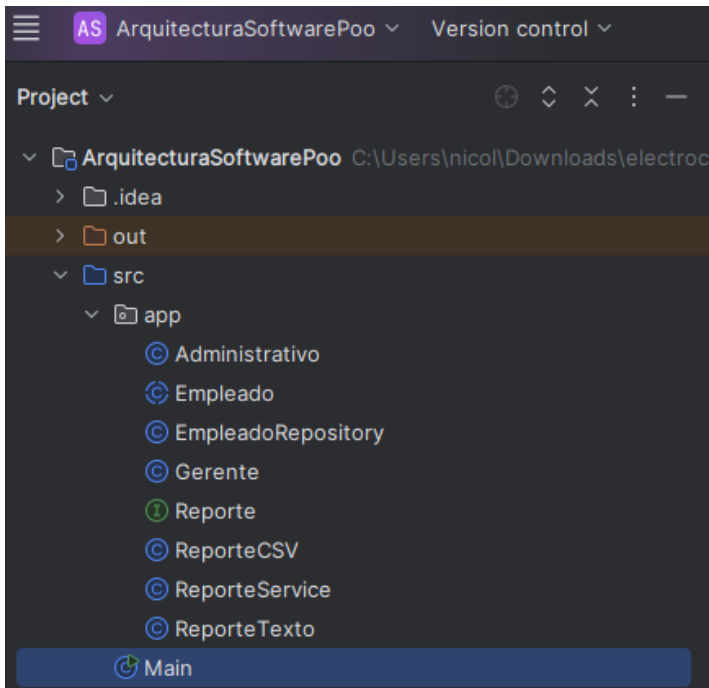


Ilustración 1 Estructura Java

Abstracción

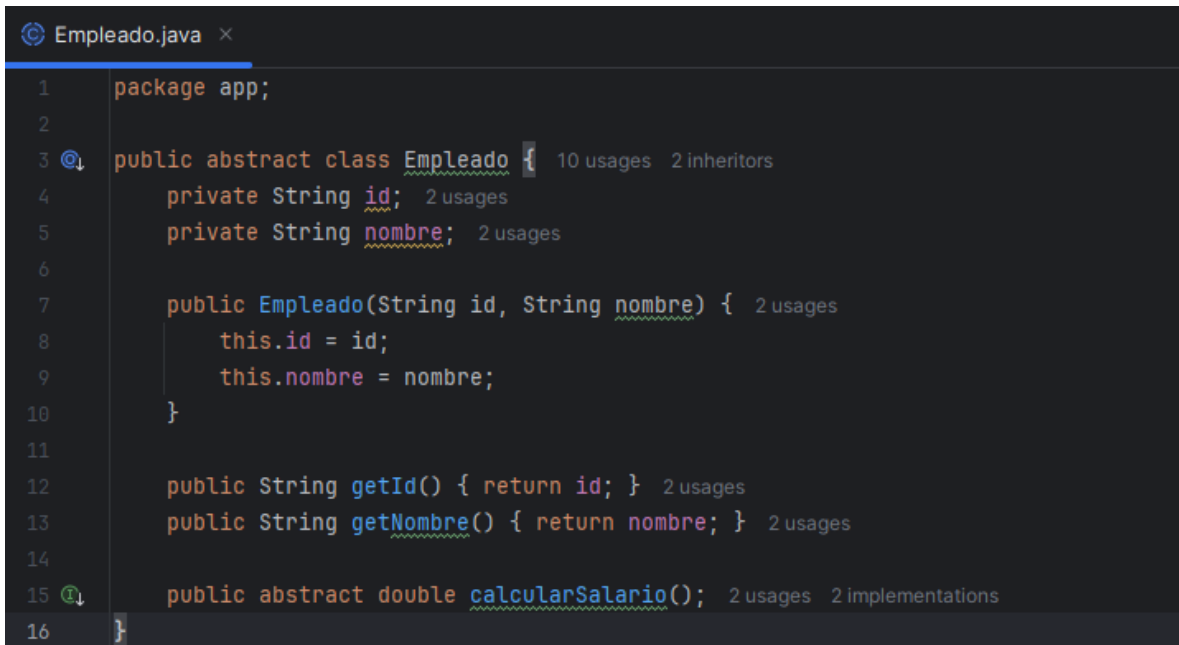
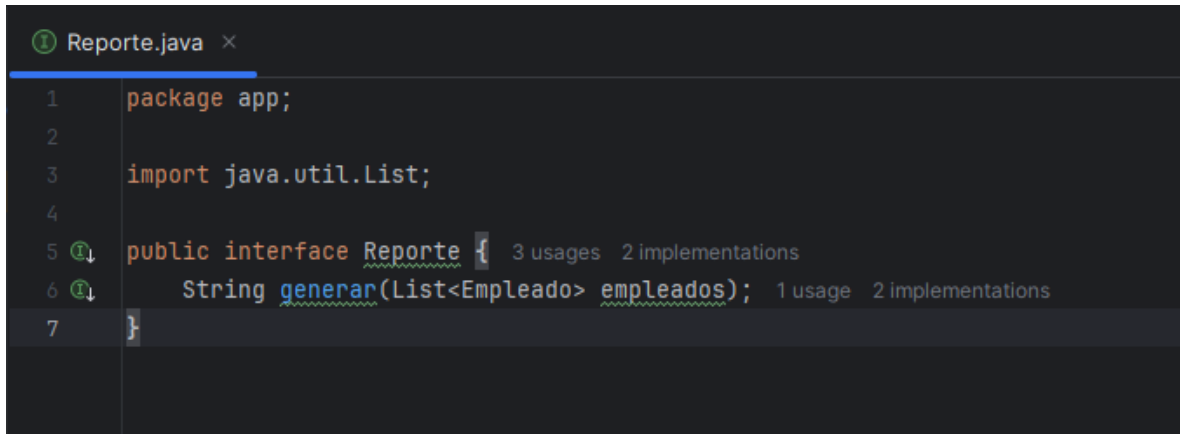


Ilustración 2 Abstracción

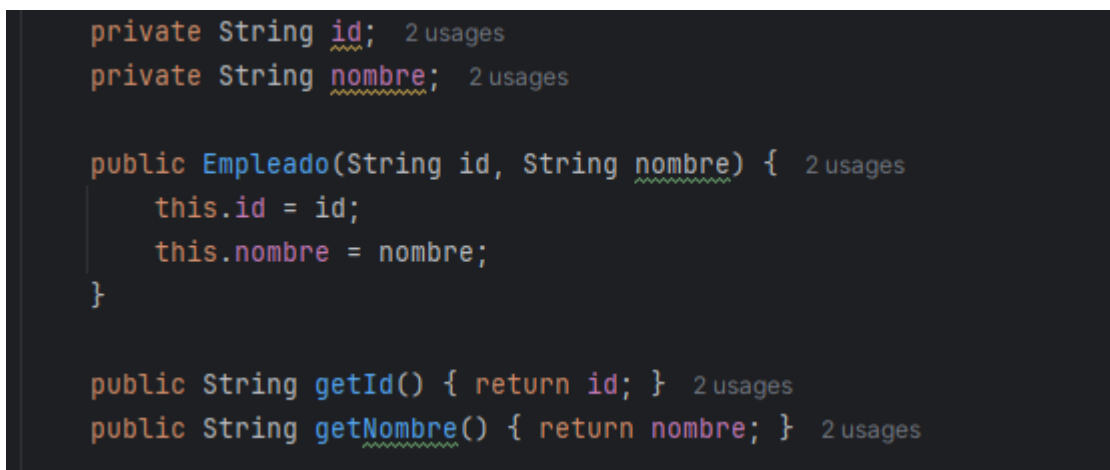
La abstracción en POO permite definir lo esencial de un concepto sin entrar en detalles específicos de su implementación. Por ejemplo, una **clase abstracta Empleado** establece las características comunes como id, nombre y un método abstracto calcularSalario(), dejando a las subclases la responsabilidad de implementar cómo se calcula dicho salario según el tipo de empleado. De manera similar, una **interfaz Reporte** define las operaciones que cualquier reporte debe cumplir, sin importar cómo se generen o presenten, garantizando flexibilidad y consistencia en el diseño del sistema.



```
1 package app;
2
3 import java.util.List;
4
5 public interface Reporte { 3 usages 2 implementations
6     String generar(List<Empleado> empleados); 1 usage 2 implementations
7 }
```

Ilustración 3 Interfaz Reporte

Encapsulamiento



```
private String id; 2 usages
private String nombre; 2 usages

public Empleado(String id, String nombre) { 2 usages
    this.id = id;
    this.nombre = nombre;
}

public String getId() { return id; } 2 usages
public String getNombre() { return nombre; } 2 usages
```

Ilustración 4 Atributos Empleados

La **encapsulación** en Java busca proteger el estado interno de una clase evitando accesos directos y no controlados a sus atributos. En el caso de la clase **Empleado**, los campos id y nombre se declaran como private, lo que impide que otras clases los modifiquen directamente. En su lugar, se proporcionan métodos públicos como getId() y getNombre(), que permiten acceder a sus valores de manera controlada. Esta práctica asegura mayor **seguridad, integridad de los datos y flexibilidad**, ya que si en el futuro se requiere validar, transformar o restringir la información, se puede hacer dentro de los getters y setters sin exponer la lógica interna al resto del sistema.

```
package app;
import java.util.ArrayList;
import java.util.List;

public class EmpleadoRepository { 4 usages
    private List<Empleado> storage = new ArrayList<>(); 2 usages

    public void add(Empleado e) { storage.add(e); } 2 usages
    public List<Empleado> findAll() { return new ArrayList<>(storage); } 1 usage
}
```

Ilustración 5 Empleado Repository

En la clase **EmpleadoRepository**, la lista storage está **encapsulada** al declararse como private, lo que impide que otras clases accedan o modifiquen directamente su contenido. En lugar de exponerla, se ofrecen métodos públicos como add() y findAll(), que permiten controlar cómo se agregan y consultan los empleados almacenados. De esta manera, se protege la integridad de los datos, se evita un uso indebido de la lista y se garantiza que las operaciones sobre ella sigan las reglas definidas en el repositorio, manteniendo un diseño más seguro y ordenado.

Herencia

```
public class Administrativo extends Empleado { 1 usage
    private double salarioMensual; 2 usages
```

Ilustración 6 Administrativo hereda Empleado

```
public class Gerente extends Empleado { 1 usage
    private double salarioBase; 2 usages
    private double bonus; 2 usages
```

Ilustración 7 Gerente hereda Empleado

La **herencia** permite crear nuevas clases basadas en una clase existente, reutilizando sus atributos y comportamientos comunes. En este caso, tanto **Administrativo** como **Gerente** heredan de la clase **Empleado**, lo que les da acceso directo a propiedades como `id` y `nombre`, así como a métodos definidos en la clase base. La diferencia radica en que cada subclase implementa su propia versión del cálculo de salario, adaptada a sus reglas específicas. Este enfoque evita la duplicación de código, facilita el mantenimiento y establece una **jerarquía clara y coherente**, donde la clase padre abstrae lo general y las clases hijas concretan lo particular.

Polimorfismo

```
public class ReporteService { 2 usages
    private EmpleadoRepository repo; 2 usages

    public ReporteService(EmpleadoRepository repo) { 1 usage
        this.repo = repo;
    }

    public String generarReporte(Reporte reporte) { 2 usages
        return reporte.generar(repo.findAll());
    }
}
```

Ilustración 8 Generar reporte

El **polimorfismo** permite que un mismo método pueda ejecutarse de distintas formas según el objeto que lo invoque. En el ejemplo de `ReporteService`, el método `generarReporte(Reporte reporte)` recibe un objeto del tipo **Reporte**, que puede ser una implementación como `ReporteTexto`, `ReporteCSV` u otra futura. Al llamar a `reporte.generar(...)`, no importa cuál sea la clase concreta, siempre se invoca el método `generar` definido en esa implementación específica. Esto significa que el mismo método produce resultados diferentes dependiendo del objeto que lo ejecute, lo que aporta **flexibilidad, escalabilidad y bajo acoplamiento** al sistema.

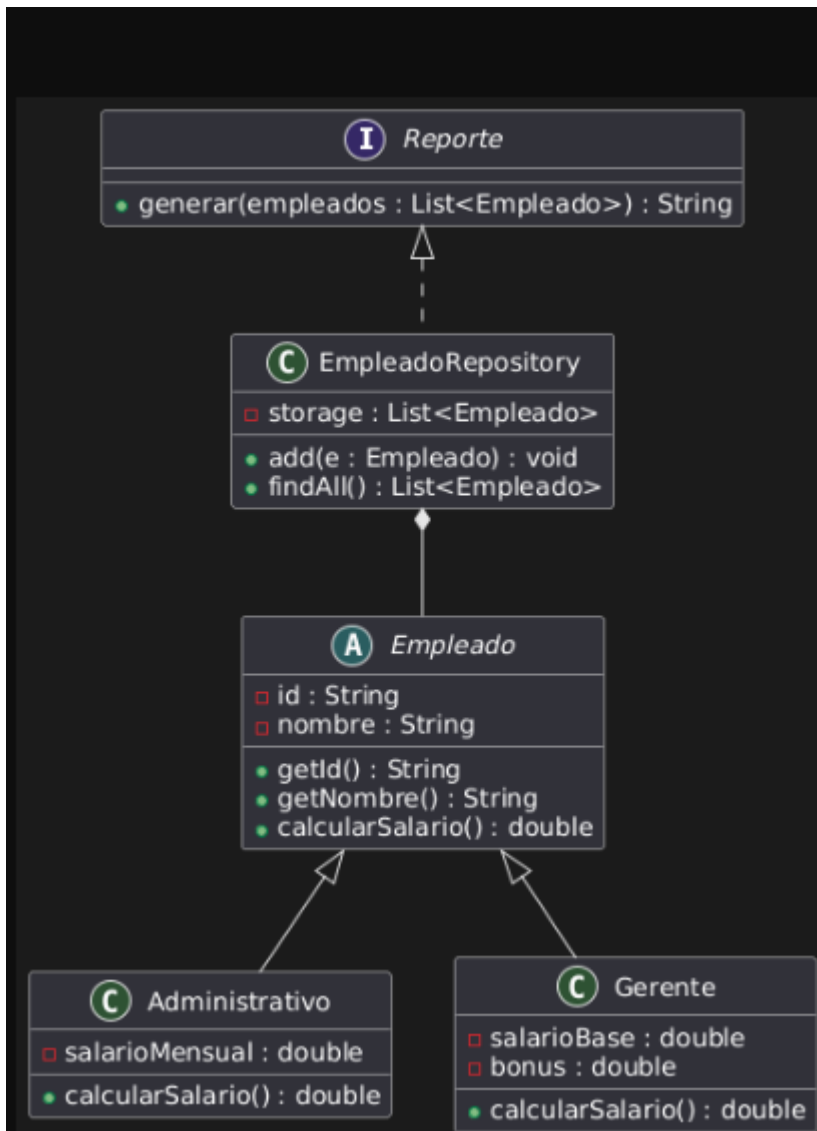


Ilustración 9 DiagramaUML

El diagrama UML ilustra cómo el sistema aplica los conceptos de la programación orientada a objetos y los principios SOLID. La interfaz **Reporte** establece un contrato general para la generación de reportes en distintos formatos, demostrando el principio de apertura a la extensión. La clase **EmpleadoRepository** se limita a la gestión de almacenamiento en memoria, lo que refleja el principio de responsabilidad única. La clase abstracta **Empleado** define los atributos y métodos comunes, mientras que las subclases **Administrativo** y **Gerente** implementan de forma particular el cálculo de salario, evidenciando el uso de herencia y polimorfismo. En conjunto, el diagrama muestra un diseño con responsabilidades bien delimitadas y una arquitectura flexible y escalable.

Principios SOLID aplicados

S — Single Responsibility Principle (SRP)

Una clase debe tener **una única responsabilidad** o motivo de cambio. Esto simplifica el mantenimiento y mejora la legibilidad del código.

O — Open/Closed Principle (OCP)

El software debe estar **abierto a la extensión pero cerrado a la modificación**, lo que permite agregar nuevas funcionalidades sin alterar el código ya existente.

L — Liskov Substitution Principle (LSP)

Las subclasses deben poder **sustituir a sus clases base** sin romper el comportamiento del programa, garantizando coherencia en la herencia y correcto uso del polimorfismo.

I — Interface Segregation Principle (ISP)

Es mejor definir **interfaces pequeñas y específicas** en lugar de una interfaz grande y genérica, para que las clases solo implementen lo que realmente necesitan.

D — Dependency Inversion Principle (DIP)

Los módulos de alto nivel deben depender de **abstracciones y no de implementaciones concretas**, favoreciendo el bajo acoplamiento, la flexibilidad y la testabilidad.

Ejemplo en Java

Principio de Responsabilidad Única (SRP)

```
public class EmpleadoRepository { 4 usages
    private List<Empleado> storage = new ArrayList<>(); 2 usages

    public void add(Empleado e) { storage.add(e); } 2 usages
    public List<Empleado> findAll() { return new ArrayList<>(storage); } 1 usage
}
```

Ilustración 10 Empleado Repository

La clase **EmpleadoRepository** es un buen ejemplo del principio **SRP (Single Responsibility Principle)**, ya que se limita exclusivamente a la **persistencia en memoria** de los empleados mediante la lista `storage` y los métodos `add()` y `findAll()`. No incluye lógica adicional como cálculos de salario, generación de reportes o reglas de negocio, lo que evita la sobrecarga de responsabilidades. Esta separación clara favorece la **cohesión**,

facilita el **mantenimiento** y reduce el riesgo de errores al modificar o extender el sistema, asegurando que cada clase tenga un rol bien definido dentro de la arquitectura.

```
public class ReporteService { 2 usages
    private EmpleadoRepository repo; 2 usages

    public ReporteService(EmpleadoRepository repo) { 1 usage
        this.repo = repo;
    }

    public String generarReporte(Reporte reporte) { 2 usages
        return reporte.generar(repo.findAll());
    }
}
```

Ilustración 11 Reporte Service

La clase **ReporteService** cumple con el principio **SRP**, ya que su única responsabilidad es **coordinar la generación de reportes** tomando los datos desde el repositorio y delegando la forma de generarlos a las implementaciones de Reporte. No se ocupa de la lógica de persistencia ni de cómo se construyen los reportes, lo que mantiene sus funciones claras y separadas. De igual manera, las clases **ReporteTexto** y **ReporteCSV** también siguen este principio, pues cada una se centra únicamente en **definir el formato específico de salida del reporte**. Al no involucrarse con el manejo de empleados ni con el almacenamiento, garantizan que cada clase tenga una sola función, lo que aumenta la cohesión y mejora la mantenibilidad del sistema.

Principio Abierto/Cerrado (OCP)

```
2
3 import java.util.List;
4
5 public interface Reporte { 3 usages 2 implementations
6     String generar(List<Empleado> empleados); 1 usage 2 implementations
7 }
```

Ilustración 12 Interfaz Reporte

La interfaz **Reporte** es un claro ejemplo del principio **OCP (Open/Closed Principle)**, ya que define un contrato estable con el método `generar(List<Empleado> empleados)`. Esto significa que, si en el futuro se requiere un nuevo formato de salida (por ejemplo, JSON, PDF o Excel), no es necesario **modificar** el código existente de la interfaz ni de las clases

que ya funcionan, sino simplemente **extender** el sistema creando nuevas implementaciones de Reporte. De esta manera, el diseño se mantiene flexible, escalable y resistente a errores, al permitir la evolución del software sin afectar lo que ya está en producción.

```
public class ReporteCSV implements Reporte { 1 usage
    @Override 1 usage
    public String generar(List<Empleado> empleados) {
        StringBuilder sb = new StringBuilder("id,nombre,salario\n");
```

Ilustración 13 Reporte CSV Implementa Reporte

```
public class ReporteTexto implements Reporte { 1 usage
    @Override 1 usage
    public String generar(List<Empleado> empleados) {
        StringBuilder sb = new StringBuilder();
        for (Empleado e : empleados) {
            sb.append(e.getId()).append(" - ").append(e.getNombre())
                .append(" : ").append(e.calcularSalario()).append("\n");
```

Ilustración 14 Reporte Texto Implementa Reporte

Las clases **ReporteTexto** y **ReporteCSV** muestran cómo aplicar en la práctica el principio **OCP (Open/Closed Principle)**. Cada una implementa la interfaz Reporte con su propio formato, y si mañana se necesita un **ReportePDF** o un **ReporteJSON**, basta con crear una nueva clase que implemente la misma interfaz. Lo importante es que no se requiere modificar ni **ReporteService** ni **EmpleadoRepository**, ya que ambos dependen de la abstracción Reporte y no de implementaciones concretas. Esto permite **extender el comportamiento** del sistema agregando nuevos reportes sin alterar el código ya probado, lo cual reduce riesgos, mejora la escalabilidad y mantiene la arquitectura estable y flexible.

¿Cómo pueden estos principios mejorar la escalabilidad y mantenibilidad del sistema en un proyecto real?

Aplicar los principios **SOLID** en un proyecto real ayuda a que el sistema sea mucho más fácil de mantener y crecer con el tiempo. Al dividir bien las responsabilidades y trabajar con abstracciones, cada parte del código se vuelve más clara y sencilla de modificar sin afectar al resto. Esto significa que cuando el negocio cambie o se necesite agregar nuevas funciones, no será necesario reescribir lo que ya funciona, sino simplemente extenderlo. Así se reducen errores, el equipo puede trabajar de manera más organizada y el sistema puede adaptarse mejor a futuro, logrando un software más estable, flexible y duradero.

Evaluación de impacto arquitectónico

Imagina que estás diseñando el backend de una aplicación de reservas de salas de reuniones.

¿Cómo te ayudaría la POO a organizar las clases y componentes?

La **POO** me ayudaría a organizar el backend de una aplicación de reservas de salas de reuniones porque permite representar de forma clara los elementos principales del sistema como objetos. Por ejemplo, podría tener una clase **Sala** con atributos como capacidad y ubicación, una clase **Reserva** con la fecha, hora y usuario que la solicita, y una clase **Usuario** con sus credenciales y permisos. Cada clase tendría sus propias responsabilidades y métodos, lo que facilita dividir el sistema en componentes más pequeños y comprensibles. Además, al usar principios como encapsulación y herencia, se puede reutilizar código común (por ejemplo, distintos tipos de usuarios que hereden de una clase base) y mantener el sistema más ordenado, flexible y fácil de mantener.

¿Qué decisiones arquitectónicas podrías tomar basándote en principios como la herencia o el modularidad?

Podría tomar decisiones arquitectónicas importantes aprovechando la **herencia** y la **modularidad**. Por ejemplo, con la herencia definiría una clase base **Usuario** con atributos comunes (nombre, correo, credenciales) y luego crearía subclases como **Administrador** o **Empleado**, cada una con permisos y comportamientos específicos, evitando repetir código. También podría usar interfaces, como un **Notificador**, que se implemente en distintas formas de envío (correo, SMS, push), manteniendo el sistema flexible ante cambios futuros.

Desde la **modularidad**, organizaría el backend en módulos separados según sus responsabilidades: un módulo de **gestión de salas** para crear y administrar espacios, un módulo de **reservas** para manejar disponibilidad y asignación de horarios, y un módulo de **notificaciones** para avisar a los usuarios. Esto reduciría el acoplamiento entre partes del sistema, haría más sencillo probar cada componente de manera independiente y permitiría escalar o modificar funcionalidades sin afectar al resto.

¿Qué riesgos podrías enfrentar si se ignoran los principios SOLID en este sistema?

Si se ignoran los principios **SOLID** en un sistema de reservas de salas, podrían surgir varios riesgos. Por ejemplo, las clases tenderían a tener demasiadas responsabilidades, volviéndose difíciles de entender y de mantener. El código estaría muy acoplado, de modo que un pequeño cambio en un módulo (como en reservas) podría romper otros (como notificaciones). Además, sería complicado **extender** el sistema con nuevas funciones, como agregar un nuevo tipo de usuario o un nuevo formato de reporte, ya que habría que

modificar código ya probado, aumentando el riesgo de errores. En conjunto, el sistema se volvería **frágil, rígido y poco escalable**, lo que encarecería su mantenimiento y limitaría su capacidad de adaptarse a las necesidades del negocio.